

NUMA DIGITAL FARM

IoT Workshop — Day 2

ESP8266 / ESP32 Networking & HTTP

Wi-Fi Connectivity · HTTP Requests · REST APIs · Web Server · JSON · Tinkercad Projects

Duration: 2 Hours | Level: Beginner → Advanced

Learning Objectives

By the end of Day 2, you will be able to:

- Connect ESP32 and ESP8266 boards to a Wi-Fi network in Station (STA) mode
- Configure the ESP as an Access Point (AP) and combined AP+STA mode
- Understand HTTP protocol — methods, status codes, request/response cycle
- Make HTTP GET requests to public APIs and receive JSON data
- Make HTTP POST requests to send sensor data to cloud endpoints
- Host a web server on the ESP to display live sensor data in a browser
- Format and parse JSON payloads for IoT communication
- Test APIs using Postman and the browser

SECTION 1

Core Concepts — From Scratch

1. Understanding Wi-Fi on ESP Microcontrollers

Both the ESP8266 and ESP32 have built-in Wi-Fi radios. This is what makes them ideal for IoT — they can connect to your home/office router and talk to the internet or local network without any external hardware.

1.1 Wi-Fi Operating Modes

Mode	Abbreviation	Description	Use Case
Station	STA	Connects TO an existing router like your phone does	Sending data to cloud, fetching APIs
Access Point	AP	BECOMES a router — other devices connect to it	Local config page, offline sensor hub
AP + Station	AP+STA	Both at same time	Mesh bridges, config portal while staying online

1.2 Connecting to Wi-Fi — Step by Step

The following is the foundational code every ESP networking program uses. Memorize this pattern.

```
#include <WiFi.h>           // Use <ESP8266WiFi.h> for ESP8266

const char* ssid    = "YourWiFiName";    // Replace with your SSID
const char* password = "YourPassword";    // Replace with your password

void setup() {
  Serial.begin(115200);
  Serial.println();
  Serial.print("Connecting to: ");
  Serial.println(ssid);

  WiFi.mode(WIFI_STA);           // Station mode (connect to router)
  WiFi.begin(ssid, password);     // Initiate connection
}
```

```
// Wait until connected (blocking poll)
while (WiFi.status() != WL_CONNECTED) {
  delay(500);
  Serial.print(".");          // Print dots while waiting
}

Serial.println();
Serial.println("WiFi Connected!");
Serial.print("IP Address: ");
Serial.println(WiFi.localIP()); // e.g. 192.168.1.105
}

void loop() {
  // Your network code goes here
}
```

Key Functions Explained

WiFi.mode(WIFI_STA) — Sets the radio to client mode. WiFi.begin(ssid, pass) — Sends the association request to the router. WiFi.status() — Returns current connection state (WL_CONNECTED = 3). WiFi.localIP() — Returns the IP the router assigned via DHCP.

1.3 Access Point (AP) Mode

In AP mode the ESP itself becomes a Wi-Fi router. Other devices (phone, laptop) can connect directly to it.

```
#include <WiFi.h>

void setup() {
  Serial.begin(115200);
  WiFi.softAP("ESP32_AP", "12345678"); // SSID, Password (min 8 chars)

  Serial.print("AP IP Address: ");
  Serial.println(WiFi.softAPIP()); // Default: 192.168.4.1
}

void loop() {}
```

1.4 Wi-Fi Status Codes Reference

Status Code	Value	Meaning
WL_CONNECTED	3	Successfully associated and authenticated

Status Code	Value	Meaning
WL_DISCONNECTED	6	Not connected (initial state)
WL_NO_SSID_AVAIL	1	SSID not found in scan results
WL_CONNECT_FAILED	4	Wrong password or auth failure
WL_IDLE_STATUS	0	Changing state, try again

2. HTTP Protocol — The Language of the Web

HTTP stands for HyperText Transfer Protocol. It is the foundation of data communication on the internet. Every time you open a website or send data from an ESP to a server, HTTP is being used.

2.1 The Request / Response Cycle

HTTP works like a conversation — the client (ESP) sends a Request, and the server sends back a Response.

HTTP Flow

Client (ESP) → [HTTP Request] → Server Client (ESP) ← [HTTP Response] ← Server
 Each request contains: Method + URL + Headers + (optional) Body
 Each response contains: Status Code + Headers + Body

2.2 HTTP Methods

Method	Purpose	IoT Use Case	Has Body?
GET	Retrieve data from server	Fetch weather, fetch config	No
POST	Send data to server	Upload sensor readings	Yes
PUT	Replace/update a resource	Update device settings	Yes
DELETE	Remove a resource	Delete a log entry	No
PATCH	Partial update	Change just one sensor value	Yes

2.3 HTTP Status Codes

Code	Meaning	What it means for your ESP
200 OK	Success	Server accepted and processed your request
201 Created	Resource created	POST was successful, new data saved
400 Bad Request	Malformed request	Check your JSON syntax or headers
401 Unauthorized	Auth required	You need an API key / token
404 Not Found	URL doesn't exist	Check your endpoint URL spelling
500 Server Error	Server crashed	Backend problem, not your ESP's fault

2.4 Understanding URLs

A URL (Uniform Resource Locator) tells HTTP where to send the request.

```
https://api.example.com:8080/data/sensors?device=esp32&limit=10
```

Protocol (http or https)

Host (domain or IP address)

Port (default 80 for HTTP, 443 for HTTPS)

Path (endpoint)

Query Parameters

3. Making HTTP GET Requests

HTTP GET is used to fetch data from a server. You use it to read APIs, fetch configurations, or pull data from databases.

3.1 Required Library

- ESP32: `HTTPClient.h` — built into the ESP32 Arduino core (no install needed)
- ESP8266: `ESP8266HTTPClient.h` — built into the ESP8266 Arduino core

3.2 Basic HTTP GET — Fetch a Public API

```
// — ESP32 HTTP GET Example —————
#include <WiFi.h>
#include <HTTPClient.h>

const char* ssid      = "YourSSID";
const char* password = "YourPassword";

void setup() {
  Serial.begin(115200);
  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) { delay(500); Serial.print("."); }
  Serial.println("\nConnected!");
}

void loop() {
  if (WiFi.status() == WL_CONNECTED) {
    HTTPClient http;

    // 1. Specify the URL
    http.begin("http://wttr.in/Kathmandu?format=3");

    // 2. Send GET request and capture response code
    int httpStatusCode = http.GET();

    // 3. Check if request was successful
    if (httpStatusCode > 0) {
      Serial.print("HTTP Response Code: ");
      Serial.println(httpStatusCode); // Should print 200

      // 4. Read the response body
      String payload = http.getString();
      Serial.println("Response: ");
      Serial.println(payload);
    } else {
      Serial.print("Error Code: ");
      Serial.println(httpStatusCode);
    }

    // 5. Always free resources
    http.end();
  }

  delay(30000); // Request every 30 seconds (be respectful to APIs!)
}
```

Important

Always call `http.end()` after every request. Failing to do this causes memory leaks and will

crash your ESP after several requests.

3.3 HTTP GET with Custom Headers (API Key)

Many real-world APIs require an API key sent in the request headers for authentication.

```
HTTPClient http;
http.begin("https://api.openweathermap.org/data/2.5/weather?q=Kathmandu");

// Add custom header for API authentication
http.addHeader("x-api-key", "your_api_key_here");
http.addHeader("Accept", "application/json");

int code = http.GET();
String response = http.getString();
http.end();
```

4. Making HTTP POST Requests

HTTP POST is used to SEND data to a server. In IoT this is how your ESP uploads sensor readings, events, or alerts to a cloud platform or your own backend.

4.1 What is JSON?

JSON (JavaScript Object Notation) is the universal data format for IoT APIs. It is lightweight, human-readable, and supported by every language and platform.

```
// JSON example – sensor data payload
{
  "device_id": "esp32_001",
  "temperature": 28.5,
  "humidity": 65,
  "gas_level": 320,
  "timestamp": 1700000000
}
```

4.2 HTTP POST with JSON Body

```
// — ESP32 HTTP POST Example —
#include <WiFi.h>
```

```
#include <HTTPClient.h>
#include <ArduinoJson.h> // Install via Library Manager: ArduinoJson by
                          Benoit Blanchon

const char* ssid      = "YourSSID";
const char* password  = "YourPassword";
const char* serverUrl = "http://192.168.1.100:3000/api/sensors"; // Your
server

// Simulate sensor values (replace with real sensor reads in projects)
float temperature = 27.4;
float humidity    = 63.0;

void setup() {
  Serial.begin(115200);
  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) { delay(500); Serial.print("."); }
  Serial.println("\nConnected!");
}

void loop() {
  if (WiFi.status() == WL_CONNECTED) {
    HTTPClient http;
    http.begin(serverUrl);

    // Required header for JSON — MUST set this!
    http.addHeader("Content-Type", "application/json");

    // Build JSON payload using ArduinoJson library
    StaticJsonDocument<200> doc;
    doc["device"]      = "ESP32_Room1";
    doc["temperature"] = temperature;
    doc["humidity"]    = humidity;
    doc["unit"]        = "celsius";

    String jsonBody;
    serializeJson(doc, jsonBody);
    Serial.println("Sending: " + jsonBody);

    // Send POST request
    int httpResponseCode = http.POST(jsonBody);

    Serial.print("Response Code: ");
    Serial.println(httpResponseCode); // 200 or 201 = success

    if (httpResponseCode > 0) {
      Serial.println("Server reply: " + http.getString());
    }
  }
}
```

```
    http.end();  
  }  
  delay(10000); // Post every 10 seconds  
}
```

4.3 Building JSON Without ArduinoJson (String Concatenation)

For simple payloads you can build JSON as a plain String — useful when you can't install extra libraries.

```
float temp = 25.3;  
float hum  = 60.1;  
  
// Manual JSON string construction  
String jsonBody = "{\"temperature\":";  
jsonBody += temp;  
jsonBody += "\",\"humidity\":";  
jsonBody += hum;  
jsonBody += "}";  
// Result: {"temperature":25.3,"humidity":60.1}  
  
http.addHeader("Content-Type", "application/json");  
http.POST(jsonBody);
```



Tip: ArduinoJson vs String concatenation

Use ArduinoJson (recommended) for production — it handles escaping, nesting, and arrays properly. Use manual String building only for quick tests with flat, simple data.

5. ESP as a Web Server

Your ESP can host a small website that any device on the same network can visit by typing the ESP's IP address into a browser. This is powerful for dashboards, configuration pages, and control panels.

5.1 How a Web Server Works

Web Server Flow

1. ESP starts the server on port 80 (default HTTP port)
2. User opens browser and types: `http://192.168.1.105`
3. Browser sends GET / HTTP/1.1 to that IP
4. ESP receives the request, runs your handler function
5. ESP sends back HTML with `http.send(200, "text/html", htmlContent)`
6. Browser renders the HTML

5.2 Basic Web Server — Sensor Display Page

```
// — ESP32 Web Server Example —————
#include <WiFi.h>
#include <WebServer.h>    // ESP32
// #include <ESP8266WebServer.h> // Use this line for ESP8266

const char* ssid      = "YourSSID";
const char* password = "YourPassword";

WebServer server(80); // Create server on port 80
// ESP8266WebServer server(80); // For ESP8266

// Simulated sensor values
float temperature = 27.4;
float humidity    = 62.0;

// — Handler: called when browser visits http://<IP>/
void handleRoot() {
  String html = "<!DOCTYPE html><html><head>";
  html += "<title>ESP Sensor Dashboard</title>";
  html += "<meta http-equiv='refresh' content='5'>"; // Auto-refresh every 5s
  html += "<style>";
  html += "  body { font-family: Arial; background: #1a1a2e; color: white;";
  html += "text-align: center; }";
  html += "  .card { background: #0077b6; border-radius: 12px; padding: 20px;";
  html += "margin: 20px auto; width: 250px; }";
  html += "  h1 { color: #00b4d8; }";
  html += "  .value { font-size: 48px; font-weight: bold; }";
  html += "</style></head><body>";
  html += "<h1>ESP32 Live Dashboard</h1>";
```

```

    html += "<div class='card'><p>Temperature</p>";
    html += "<p class='value'>" + String(temperature, 1) + " &deg;C</p></div>";
    html += "<div class='card'><p>Humidity</p>";
    html += "<p class='value'>" + String(humidity, 1) + " %</p></div>";
    html += "</body></html>";

    server.send(200, "text/html", html);
}

// — Handler: 404 for unknown routes
void handleNotFound() {
    server.send(404, "text/plain", "404: Page Not Found");
}

// — Handler: JSON API endpoint
void handleAPI() {
    String json = "{\"temperature\":";
    json += temperature;
    json += ", \"humidity\":";
    json += humidity;
    json += "}";
    server.send(200, "application/json", json);
}

void setup() {
    Serial.begin(115200);
    WiFi.begin(ssid, password);
    while (WiFi.status() != WL_CONNECTED) { delay(500); Serial.print("."); }
    Serial.println("\nConnected! IP: " + WiFi.localIP().toString());

    server.on("/", handleRoot);    // Register routes
    server.on("/api", handleAPI);
    server.onNotFound(handleNotFound);

    server.begin();                // Start listening
    Serial.println("Server started. Open http://" + WiFi.localIP().toString());
}

void loop() {
    server.handleClient(); // CRITICAL: must call every loop to process
    requests
}

```

⚠ **CRITICAL: server.handleClient()**

You **MUST** call `server.handleClient()` in every `loop()` iteration. Without this, the ESP will never respond to incoming browser requests. Think of it as the ESP's task to check for incoming mail — you must check repeatedly.

5.3 Web Server Routes Reference

Route	Method	Handler Example	Purpose
server.on("/",...)	GET	handleRoot()	Main dashboard page
server.on("/data",...)	GET	handleData()	Return JSON sensor data
server.on("/led/on",...)	GET	handleLedOn()	Control LED via URL
server.on("/led/off",...)	GET	handleLedOff()	Control LED via URL
server.onNotFound(...)	ANY	handleNotFound()	Catch-all 404 handler

6. REST APIs and Testing with Postman

6.1 What is a REST API?

REST (Representational State Transfer) is a design style for web APIs. A REST API is a web service that accepts HTTP requests and returns data (usually JSON).

- Each URL represents a resource (e.g. /sensors, /devices, /alerts)
- HTTP methods define the action: GET=read, POST=create, PUT=update, DELETE=remove
- Stateless: each request must carry all needed information
- Returns standard HTTP status codes to indicate success or failure

6.2 Testing APIs with Postman

Postman is a free desktop app that lets you send any HTTP request to any server and inspect the response. Use it to test your ESP server endpoints and external APIs before writing code.

Step	Action in Postman	Notes
1	Open Postman, click New Request	Free download at postman.com
2	Select method: GET / POST	From dropdown on left
3	Enter URL	e.g. <code>http://192.168.1.105/api</code>
4	For POST: click Body → raw → JSON	Paste your JSON payload
5	Click Send	Response appears in lower pane

Step	Action in Postman	Notes
6	Check Status Code and Body	Should match what your ESP sends

6.3 Public APIs for Testing

API	URL	What it Returns
Weather (text)	<code>http://wttr.in/YourCity?format=3</code>	City weather as plain text
IP Info	<code>http://ip-api.com/json</code>	Your public IP and location
Joke API	<code>https://icanhazdadjoke.com</code>	Random dad joke
HTTPBin POST	<code>https://httpbin.org/post</code>	Echoes back your POST body — great for testing
JSONPlaceholder	<code>https://jsonplaceholder.typicode.com/posts</code>	Fake REST API for testing

SECTION 2

Complete Projects with Pin Configurations

Project 1 — Wi-Fi Scanner

Difficulty: ★☆☆ | Hardware: ESP32 or ESP8266 only | Duration: 15 minutes

This project scans all available Wi-Fi networks nearby and prints their SSID, signal strength (RSSI in dBm), and encryption type to the Serial Monitor. Great for understanding the wireless environment.

Pin Configuration

No external components needed — only the ESP32/ESP8266 module and USB cable.

Component	Connection	Notes
ESP32 / ESP8266	USB to computer	Powers the board and provides Serial Monitor

Project 1 Code

```
// — Project 1: Wi-Fi Network Scanner —
#include <WiFi.h>    // Change to <ESP8266WiFi.h> for ESP8266

void setup() {
  Serial.begin(115200);
  delay(1000);

  WiFi.mode(WIFI_STA);    // Must be in STA mode to scan
  WiFi.disconnect();      // Ensure not connected to any AP
  delay(100);

  Serial.println("=== Wi-Fi Scanner Ready ===");
}

void loop() {
  Serial.println("\nScanning...");

  // Scan for networks. 'true' = async (non-blocking)
  int n = WiFi.scanNetworks();

  if (n == 0) {
    Serial.println("No networks found.");
  } else {
```

```
Serial.println(String(n) + " networks found:");
Serial.println("_____");

for (int i = 0; i < n; i++) {
  // Print network info
  Serial.print(i + 1);
  Serial.print(". SSID: ");
  Serial.print(WiFi.SSID(i));

  // Print signal strength
  Serial.print(" RSSI: ");
  Serial.print(WiFi.RSSI(i));
  Serial.print(" dBm");

  // Print encryption
  Serial.print(" Enc: ");
  Serial.println(WiFi.encryptionType(i) == WIFI_AUTH_OPEN ? "Open" :
"Encrypted");
  delay(10);
}
Serial.println("_____");
}

delay(10000); // Scan every 10 seconds
}
```

Expected Serial Monitor Output

```
Scanning...
5 networks found:
_____
1. SSID: HomeRouter      RSSI: -45 dBm  Enc: Encrypted
2. SSID: Office_WiFi    RSSI: -67 dBm  Enc: Encrypted
3. SSID: GuestNetwork   RSSI: -72 dBm  Enc: Open
_____
```

Project 2 — Live Weather Fetcher (HTTP GET)

Difficulty: ★★☆☆ | Hardware: ESP32 + 0 external parts | Duration: 25 minutes

Connects to Wi-Fi and fetches live weather data for your city from a public API every 30 seconds. Prints formatted weather to the Serial Monitor.

Pin Configuration

Component	ESP32 Pin	ESP8266 Pin	Notes
USB Cable	USB port	USB port	Power + programming
None	—	—	All networking is internal radio

Project 2 Code

```
// — Project 2: Live Weather via HTTP GET —————
#include <WiFi.h>
#include <HTTPClient.h>

const char* ssid      = "YourSSID";
const char* password  = "YourPassword";
const char* city       = "Kathmandu"; // Change to your city

String buildWeatherURL() {
    return String("http://wttr.in/") + city + "?format=%l:+%C+%t+Feels+
%f+Humidity+%h";
}

void fetchWeather() {
    HTTPClient http;
    http.begin(buildWeatherURL());
    http.setTimeout(8000); // 8 second timeout

    int code = http.GET();

    if (code == 200) {
        String weather = http.getString();
        Serial.println("\n Weather Update:");
        Serial.println("_____");
        Serial.println(weather);
        Serial.println("_____");
    } else {
        Serial.println("HTTP Error: " + String(code));
    }
    http.end();
}

void setup() {
    Serial.begin(115200);
    WiFi.begin(ssid, password);
    Serial.print("Connecting");
    while (WiFi.status() != WL_CONNECTED) { delay(500); Serial.print("."); }
```

```
    Serial.println("\nConnected! IP: " + WiFi.localIP().toString());
    fetchWeather(); // Fetch immediately on boot
}

void loop() {
    fetchWeather();
    delay(30000); // Every 30 seconds
}
```

Project 3 — DHT11 Data Logger via HTTP POST

Difficulty: ★★☆☆ | Hardware: ESP32 + DHT11 sensor | Duration: 35 minutes

Reads real temperature and humidity from a DHT11 sensor and POSTs the data as JSON to a test server (httpbin.org mirrors your payload back so you can confirm it). Install DHT library via Library Manager.

Pin Configuration

DHT11 Pin	ESP32 GPIO	ESP8266 GPIO	Note
VCC (pin 1)	3.3V or 5V	3.3V	Both voltages work
DATA (pin 2)	GPIO 4 (D4)	GPIO 4 (D2)	Add 10kΩ pull-up to VCC
NC (pin 3)	—	—	Not connected
GND (pin 4)	GND	GND	Common ground

Wiring Diagram (Breadboard)

DHT11 Wiring Steps

1. Insert DHT11 into breadboard with pins facing you
2. Connect left pin (VCC) → 3.3V rail
3. Connect second pin (DATA) → GPIO 4 on ESP32
4. Place 10kΩ resistor between DATA pin and VCC rail (pull-up)
5. Connect right pin (GND) → GND rail
6. Connect ESP32 GND → GND rail, ESP32 3.3V → 3.3V rail

Project 3 Code

```
// — Project 3: DHT11 Sensor → HTTP POST —
#include <WiFi.h>
#include <HTTPClient.h>
#include <DHT.h>           // Install: DHT sensor library by Adafruit
#include <ArduinoJson.h>    // Install: ArduinoJson by Benoit Blanchon

#define DHTPIN 4           // GPIO 4 — DATA pin of DHT11
#define DHTTYPE DHT11      // DHT11 or DHT22

const char* ssid = "YourSSID";
const char* password = "YourPassword";

// httpbin.org echoes POST body back — perfect for testing without a server
const char* serverUrl = "https://httpbin.org/post";
```

```
DHT dht(DHTPIN, DHTTYPE);

void postSensorData(float temp, float hum) {
  HTTPClient http;
  http.begin(serverUrl);
  http.addHeader("Content-Type", "application/json");

  // Build JSON payload
  StaticJsonDocument<256> doc;
  doc["device"]      = "ESP32_DHT11";
  doc["temperature"] = serialized(String(temp, 1));
  doc["humidity"]    = serialized(String(hum, 1));
  doc["unit"]        = "celsius";

  String jsonStr;
  serializeJson(doc, jsonStr);

  Serial.println("Sending JSON: " + jsonStr);

  int code = http.POST(jsonStr);

  if (code > 0) {
    Serial.println("POST success! Response code: " + String(code));
    String response = http.getString();
    // httpbin returns the payload under 'data' field
    Serial.println("Server echoed: " + response.substring(0, 200));
  } else {
    Serial.println("POST failed. Error: " + String(code));
  }
  http.end();
}

void setup() {
  Serial.begin(115200);
  dht.begin();

  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) { delay(500); Serial.print("."); }
  Serial.println("\nConnected! IP: " + WiFi.localIP().toString());
}

void loop() {
  delay(2000); // DHT11 needs 2 sec between readings

  float temp = dht.readTemperature();
  float hum  = dht.readHumidity();
}
```

```
// Check if readings are valid
if (isnan(temp) || isnan(hum)) {
  Serial.println("ERROR: Failed to read DHT11. Check wiring!");
  return; // Skip this cycle
}

Serial.printf("Temp: %.1f°C Humidity: %.1f%%\n", temp, hum);

if (WiFi.status() == WL_CONNECTED) {
  postSensorData(temp, hum);
} else {
  Serial.println("WiFi disconnected! Reconnecting...");
  WiFi.reconnect();
}

delay(8000); // Wait 8 more seconds (total 10s interval)
}
```

Project 4 — Live Sensor Web Dashboard

Difficulty: ★★★ | Hardware: ESP32 + DHT11 | Duration: 45 minutes

The ESP32 reads DHT11 sensor data and hosts a styled web dashboard. Open a browser on any device on the same network and navigate to the ESP's IP to see live readings auto-refreshed every 5 seconds.

Pin Configuration

Component	ESP32 GPIO	Wire Color (suggested)	Notes
DHT11 VCC	3.3V	Red	DHT11 power
DHT11 DATA	GPIO 4	Yellow	Add 10kΩ pull-up resistor
DHT11 GND	GND	Black	Common ground
LED (optional)	GPIO 2	Green	Built-in LED — blinks on data read
LED Resistor	GPIO 2 → LED → GND	—	220Ω current limiter

Project 4 Code

```
// — Project 4: Web Dashboard with DHT11 —
#include <WiFi.h>
#include <WebServer.h>
#include <DHT.h>

#define DHTPIN 4
#define DHTTYPE DHT11
#define LED_PIN 2 // Built-in LED on most ESP32 boards

const char* ssid = "YourSSID";
const char* password = "YourPassword";

DHT dht(DHTPIN, DHTTYPE);
WebServer server(80);

float lastTemp = 0.0;
float lastHum = 0.0;

// — HTML Dashboard (embedded in program memory)
String buildDashboard() {
  String html = R"rawhtml(
<!DOCTYPE html>
<html lang='en'>
<head>
  <meta charset='UTF-8'>
  <meta name='viewport' content='width=device-width, initial-scale=1.0'>
  <title>ESP32 Dashboard</title>
  <style>
    * { box-sizing: border-box; margin: 0; padding: 0; }
    body { font-family: Arial, sans-serif; background: #1a1a2e; color:
white; }
    header { background: #0077b6; padding: 20px; text-align: center; }
    header h1 { font-size: 28px; }
    header p { color: #90e0ef; font-size: 14px; }
    .grid { display: flex; justify-content: center; gap: 30px;
      flex-wrap: wrap; padding: 40px 20px; }
    .card { background: #16213e; border-radius: 16px; padding: 30px;
      width: 200px; text-align: center; box-shadow: 0 4px 20px
#00000080; }
    .card .icon { font-size: 48px; }
    .card .value { font-size: 52px; font-weight: bold; color: #00b4d8;
margin: 10px 0; }
    .card .label { color: #aaa; font-size: 14px; text-transform:
uppercase; }
    .status { text-align: center; color: #52b788; padding-bottom: 20px; }
  </style>
  <meta http-equiv='refresh' content='5'>

```

```

</head>
<body>
  <header>
    <h1> ESP32 Sensor Dashboard</h1>
    <p>Auto-refreshes every 5 seconds</p>
  </header>
  <div class='grid'>
)rawhtml";

// Inject live sensor values
html += "<div class='card'>";
html += "<div class='icon'>🌡️</div>";
html += "<div class='value'>" + String(lastTemp, 1) + "°C</div>";
html += "<div class='label'>Temperature</div></div>";

html += "<div class='card'>";
html += "<div class='icon'>💧</div>";
html += "<div class='value'>" + String(lastHum, 1) + "%</div>";
html += "<div class='label'>Humidity</div></div>";

html += "</div>"; // close .grid
html += "<p class='status'>Last reading from DHT11 sensor (GPIO 4)</p>";
html += "</body></html>";
return html;
}

// — JSON API endpoint – returns sensor data as JSON
void handleAPI() {
  String json = "{\"temperature\":";
  json += String(lastTemp, 2);
  json += ", \"humidity\":";
  json += String(lastHum, 2);
  json += ", \"unit\": \"celsius\"}";
  server.send(200, "application/json", json);
}

void setup() {
  Serial.begin(115200);
  pinMode(LED_PIN, OUTPUT);
  dht.begin();

  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) { delay(500); Serial.print("."); }

  Serial.println("\n Connected!");
  Serial.println("Dashboard → http://" + WiFi.localIP().toString());
  Serial.println("JSON API → http://" + WiFi.localIP().toString() + "/api");
}

```

```

server.on("/",    [](){ server.send(200, "text/html", buildDashboard()); });
server.on("/api", handleAPI);
server.onNotFound([](){server.send(404,"text/plain","Not Found"); });
server.begin();
}

void loop() {
  // Read sensor every 2 seconds
  float t = dht.readTemperature();
  float h = dht.readHumidity();

  if (!isnan(t) && !isnan(h)) {
    lastTemp = t;
    lastHum  = h;
    // Blink LED to indicate successful read
    digitalWrite(LED_PIN, HIGH); delay(50); digitalWrite(LED_PIN, LOW);
  }

  server.handleClient(); // Handle browser requests
  delay(2000);
}

```

SECTION 3

Tinkercad Virtual Prototyping Activities

Tinkercad for IoT — Getting Started

Tinkercad Circuits (tinkercad.com) is a free browser-based circuit simulator. Since Tinkercad does not support Wi-Fi simulation, Day 2 activities simulate the sensor + microcontroller portion of each project while you prototype the network code on a real board.

Tinkercad Limitation & Workaround

Tinkercad simulates Arduino UNO / Nano, not ESP32/ESP8266. For Day 2 we use: • Tinkercad → Simulate sensor circuits and Arduino programming logic (no Wi-Fi) • Real ESP board → Upload the final networking code This is the professional workflow: prototype hardware logic first, then add networking.

Tinkercad Activity 1 — Simulate DHT11 Sensor Readings

Build the DHT11 circuit in Tinkercad and verify sensor reading logic before connecting to Wi-Fi.

Step-by-Step Instructions

1. Go to tinkercad.com → Sign in → Create → Circuits
2. From the Components panel, search for 'Arduino Uno' and drag to canvas
3. Search 'TMP36' sensor (Tinkercad approximation for DHT11) — drag to canvas
4. Wire: TMP36 +V → 5V on Arduino, TMP36 GND → GND on Arduino
5. Wire: TMP36 OUT → A0 pin on Arduino (analog read pin)
6. Add an LED: LED anode (+) → Pin 9 via 220Ω resistor → GND (cathode)
7. Open Code Editor → Choose 'Text' mode → Paste the code below
8. Click Simulate → Start Simulation to see the LED blink and serial output

Tinkercad Activity 1 Code (Arduino UNO)

```
// Tinkercad Activity 1 – Read temperature sensor, control LED
// Note: TMP36 reads analog voltage; on real ESP32 use DHT11 with DHT library

const int SENSOR_PIN = A0; // TMP36 analog output
const int LED_PIN     = 9;  // LED pin
const float ALERT_TEMP = 30.0; // Alert threshold in Celsius
```

```
void setup() {
  Serial.begin(9600);
  pinMode(LED_PIN, OUTPUT);
  Serial.println("=== Temperature Monitor Started ===");
}

float readTempC() {
  int raw = analogRead(SENSOR_PIN);
  float voltage = raw * (5.0 / 1023.0); // Convert 10-bit to voltage
  float tempC = (voltage - 0.5) * 100.0; // TMP36 formula
  return tempC;
}

void loop() {
  float temp = readTempC();

  Serial.print("Temperature: ");
  Serial.print(temp, 1);
  Serial.println(" C");

  // LED ON if temperature exceeds threshold
  if (temp > ALERT_TEMP) {
    digitalWrite(LED_PIN, HIGH);
    Serial.println(" ALERT: High temperature!");
  } else {
    digitalWrite(LED_PIN, LOW);
  }

  delay(1000);
}
```

Tinkercad Activity 2 — Multi-Sensor Dashboard Simulation

Simulate an LDR light sensor and push button alongside temperature monitoring — mimicking a multi-sensor IoT node before connecting to HTTP.

Component List for Tinkercad

Component	Tinkercad Name	Arduino Pin	Purpose
Temperature Sensor	TMP36	A0	Temperature reading
Light Sensor	Photoresistor	A1	Light level reading
Push Button	Pushbutton	Pin 7	Trigger manual reading
Red LED	LED	Pin 13	Temperature alert

Component	Tinkercad Name	Arduino Pin	Purpose
			indicator
Green LED	LED	Pin 12	Normal status indicator
10kΩ Resistor (×2)	Resistor 10k	Pull-down on btn, LDR divider	Signal conditioning
220Ω Resistor (×2)	Resistor 220	LED current limiters	LED protection

Tinkercad Activity 2 Code

```
// Tinkercad Activity 2 – Multi-sensor simulation
const int TEMP_PIN    = A0;
const int LDR_PIN     = A1;
const int BUTTON_PIN  = 7;
const int RED_LED     = 13;
const int GREEN_LED   = 12;

int lastButtonState = LOW;
bool manualRead = false;

void setup() {
  Serial.begin(9600);
  pinMode(RED_LED, OUTPUT);
  pinMode(GREEN_LED, OUTPUT);
  pinMode(BUTTON_PIN, INPUT);
  Serial.println("Multi-Sensor Monitor Ready");
  Serial.println("Press button for instant reading");
}

float readTempC() {
  float v = analogRead(TEMP_PIN) * (5.0 / 1023.0);
  return (v - 0.5) * 100.0;
}

String getLightLevel(int raw) {
  if (raw < 200) return "DARK";
  if (raw < 600) return "DIM";
  return "BRIGHT";
}

void printSensorData() {
  float temp    = readTempC();
  int  ldrRaw   = analogRead(LDR_PIN);
  String light  = getLightLevel(ldrRaw);

  // Simulate JSON-style output (mirrors what ESP would POST)
  Serial.println("--- Sensor Reading ---");
}
```

```

    Serial.print("  Temperature: "); Serial.print(temp, 1); Serial.println("
C");
    Serial.print("  Light Raw:   "); Serial.println(ldrRaw);
    Serial.print("  Light Level: "); Serial.println(light);
    Serial.println("  [JSON]:
{"temp\":"+String(temp,1)+"","\light\":"+String(light)+"\"}");

    // Control LEDs based on readings
    if (temp > 28.0) {
        digitalWrite(RED_LED, HIGH);
        digitalWrite(GREEN_LED, LOW);
    } else {
        digitalWrite(RED_LED, LOW);
        digitalWrite(GREEN_LED, HIGH);
    }
}

void loop() {
    int btn = digitalRead(BUTTON_PIN);

    // Button press triggers immediate reading
    if (btn == HIGH && lastButtonState == LOW) {
        Serial.println(">>> BUTTON PRESSED – Manual Read!");
        printSensorData();
    }
    lastButtonState = btn;

    // Auto reading every 5 seconds
    static unsigned long lastTime = 0;
    if (millis() - lastTime >= 5000) {
        printSensorData();
        lastTime = millis();
    }
}

```

SECTION 4

Assignments & Challenges

Assignment 1 — Wi-Fi Connection with Reconnect Logic

★☆☆ Beginner | Expected time: 20 minutes

Objective: Write a sketch that connects to Wi-Fi and automatically reconnects if the connection drops.

Requirements:

9. Connect to Wi-Fi and print the assigned IP address
10. In loop(), check WiFi.status() every 5 seconds
11. If disconnected: print warning, call WiFi.reconnect(), wait up to 10 seconds
12. Print 'Reconnected!' when link is restored
13. BONUS: Also print uptime (millis()/1000 seconds) each loop

Assignment 2 — HTTP GET: Parse JSON Response

★★☆ Intermediate | Expected time: 30 minutes

Objective: Fetch data from <https://jsonplaceholder.typicode.com/todos/1> and parse the JSON response to extract individual fields.

Requirements:

14. Make an HTTP GET request to the URL above
15. Install and use ArduinoJson to parse the response
16. Print: userId, id, title, and completed fields separately
17. Change the URL to /todos/5 and /todos/10 — observe different responses
18. BONUS: Fetch /users/1 and extract name, email, and city from nested JSON

Hint: `DynamicJsonDocument doc(1024); deserializeJson(doc, payload);` then `doc["title"].as<String>()`

Assignment 3 — Web Server with LED Control

★★☆ Intermediate | Expected time: 35 minutes

Objective: Build a web server with two routes that turn an external LED ON and OFF from a browser.

Circuit:

- LED connected to GPIO 5 via 220Ω resistor → GND

Requirements:

19. / → Main page with two clickable links: 'Turn LED ON' and 'Turn LED OFF'
20. /led/on → Turn GPIO 5 HIGH, show confirmation page
21. /led/off → Turn GPIO 5 LOW, show confirmation page
22. /status → Return JSON: {"led": "on"} or {"led": "off"}
23. BONUS: Style the main page with a color-coded LED status indicator (green=on,

red=off)

Assignment 4 — Advanced: Environment Monitor Dashboard

★★★ Advanced | Expected time: 60–90 minutes

Objective: Build a complete IoT node that reads DHT11, serves a styled dashboard, provides a REST API, and POSTs data to httpbin.org every 60 seconds.

Requirements:

24. Read DHT11 (GPIO 4) for temperature and humidity
25. Host a web dashboard at / showing live values with auto-refresh
26. REST endpoint at /api/data returns: {temp, humidity, heatIndex, uptime_s}
27. POST a reading to <https://httpbin.org/post> every 60 seconds
28. Built-in LED blinks 3 times after each successful POST
29. Dashboard shows POST success/fail status in a colored badge
30. BONUS: Calculate and display Heat Index using: $HI = T - 0.55 \cdot (1 - H/100) \cdot (T - 14.5)$

Evaluation Criteria: Working dashboard (30%) | JSON API accuracy (20%) | Successful POST with correct payload (30%) | Code quality & comments (20%)

QUICK REFERENCE

Day 2 Cheat Sheet

Wi-Fi Functions

Function	Library	Purpose
WiFi.mode(WIFI_STA)	WiFi.h	Set to station (client) mode
WiFi.begin(ssid, pass)	WiFi.h	Connect to a router
WiFi.status()	WiFi.h	Check connection state
WiFi.localIP()	WiFi.h	Get assigned IP address
WiFi.softAP(ssid, pass)	WiFi.h	Start Access Point mode
WiFi.softAPIP()	WiFi.h	Get AP IP (default 192.168.4.1)
WiFi.RSSI()	WiFi.h	Signal strength in dBm
WiFi.reconnect()	WiFi.h	Reconnect after drop

HTTP Client Functions

Function	Purpose
http.begin(url)	Set the target URL
http.addHeader(key, val)	Add a request header (e.g. Content-Type)
http.GET()	Send GET request, returns status code
http.POST(body)	Send POST with body, returns status code
http.getString()	Read response body as String
http.end()	Free resources — ALWAYS call this
http.setTimeout(ms)	Set connection timeout in milliseconds

Web Server Functions

Function	Purpose
WebServer server(80)	Create server on port 80
server.on(path, fn)	Register a route handler function
server.onNotFound(fn)	Register 404 handler
server.begin()	Start listening for connections
server.handleClient()	Process pending requests — call in loop()
server.send(code, type, body)	Send HTTP response

Function	Purpose
<code>server.arg(name)</code>	Get URL query parameter value

Common Errors & Fixes

Error / Symptom	Likely Cause	Fix
Dots never stop printing	Wrong SSID or password	Double-check credentials, case-sensitive
HTTP Error -1	Cannot reach host	Check URL, ping host from PC
HTTP Error -11	Connection refused	Server not running or wrong port
<code>isnan(temp) == true</code>	DHT11 wiring issue	Check 10kΩ pull-up, correct pin number
Browser shows nothing	Forgot <code>server.handleClient()</code>	Add to every <code>loop()</code> iteration
Crash after a few requests	Missing <code>http.end()</code>	Always call <code>http.end()</code> after each request
JSON parse fails	Response not valid JSON	Print raw response first with <code>Serial.println</code>